Lesson 8: Under Pressure

Godot Engine

VIDEO GAME CREATION

This week: give the enemies you dodge last week actual stakes. An enemy reaching the bottom now costs a life - lose all 3, and "Game Over" appears.

Open your own [YourName]Game project from Lesson 7 - don't start a new one.

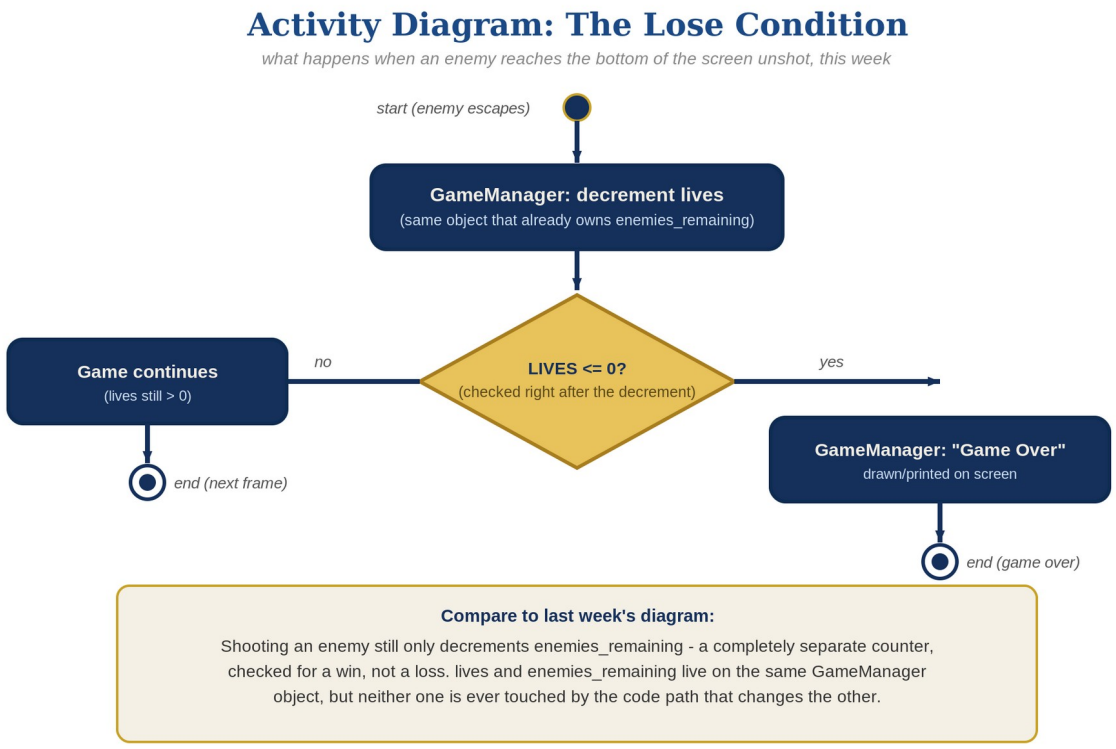
Tip: the core exercise needs no new assets - this reuses last week's escape-cleanup code and just gives it a real consequence. For Level 3 sound, use the provided sfx_lifelost.wav and sfx_gameover.wav from the Lesson 8 Assets folder.

Setup

1. Open your [YourName]Game project from Lesson 7.
2. Add a Label node to your main scene, rename it LoseText, set its Text to "Game Over", and hide it (uncheck Visible in the Inspector).
3. In enemy.tscn's own script, add a new signal at the top, alongside the existing destroyed signal: signal escaped.

Level 1 - Minimal (MVP)

Every enemy that reaches the bottom unshot now costs the player one life, tracked starting at 3. The moment lives reach zero, "Game Over" appears on screen.



This is the payoff of Rule 13's dedicated GameManager: a second piece of shared state slotted in without touching anything else.

The lose-condition logic behind this week's exercise - the same shape in every engine, whatever the exact code or blocks look like.

4. In enemy.tscn's _process(), find the escape branch you built last week. Right before queue_free(), add escaped.emit() (code below).
5. In GameManager's script - the same node that already owns enemies_remaining and on_enemy_destroyed() - add var lives := 3 and a new function on_enemy_escaped() that decrements it and shows LoseText at zero (code below).
6. In the main scene's spawn loop, right where you connect each enemy's destroyed signal to $GameManager.on_enemy_destroyed, also connect its escaped signal to $GameManager.on_enemy_escaped.
7. Run the scene and let all 3 lives run out by letting enemies escape unshot - "Game Over" should appear the instant the third one escapes.

Highlighted lines are new this week; everything else is code you already have.

```
# enemy.tscn's script:
signal escaped

func _process(delta):
    position.y += enemy_speed * delta
    if position.y > 700:
        escaped.emit()
        queue_free()

# GameManager's script - add alongside enemies_remaining and
on_enemy_destroyed():
var lives := 3

func on_enemy_escaped():
```

```
    lives -= 1
    if lives <= 0:
        get_node("../LoseText").show()

# Main scene's script - in the spawn loop, alongside connecting destroyed:
new_enemy.escaped.connect($GameManager.on_enemy_escaped)
```

Level 2 - Complete (Alpha/Beta)

Shooting an enemy never costs a life, and letting one escape never counts as a kill - the two counters stay completely independent.

- Shoot an enemy and confirm lives does NOT change - only the destroyed signal, and enemies_remaining, should be involved.
- Let an enemy escape and confirm enemies_remaining does NOT change - only the escaped signal, and lives, should be involved.
- Confirm LoseText appears exactly once, exactly when lives reaches zero.

Level 3 - Refined (Ready to Publish) - pick any that interest you

- Visual: add a third Label showing the current lives count, updated inside GameManager's on_enemy_escaped() - a nice touch, though the full scoreboard-style UI is coming properly in Lesson 9.
- Sound: add two AudioStreamPlayer nodes (not 2D, since they're UI, not positional) as children of GameManager, assign the provided sfx_lifelost.wav to one and sfx_gameover.wav to the other, then call .play() on the first inside on_enemy_escaped() right after lives decreases - and on the second instead, in the branch where lives reaches zero.
- Structure: notice that escaped and destroyed are two completely separate signals, each connected once at spawn time, each handled by its own function on GameManager - this is the same pattern that made Godot's Lesson 7 bugfix unnecessary, now extended to a second signal and kept on the same dedicated manager node introduced in Lesson 6.

If Something's Wrong

- Lives decreases when you shoot an enemy: confirm _on_hitbox_area_entered() only emits destroyed, and _process()'s escape branch only emits escaped - the two should never call into each other.
- "Game Over" never appears: confirm the spawn loop actually connects new_enemy.escaped.connect($GameManager.on_enemy_escaped) for every spawned enemy, not just destroyed.
- Nothing happens when enemies escape: confirm on_enemy_escaped() is actually defined on GameManager's script (not accidentally left on the main scene's own script), and that LoseText's node path ($../LoseText, referenced from GameManager) matches exactly.

Want More?

Student Resource List > Godot section, or docs.godotengine.org